MULTI AGENT AI EXECUTION AND RESOURCE MANAGEMENT SYSTEM

M. LIKHITHA – 2520030575

K. VAISHNAVI – 2520030198

ABSTRACT

AURA – Autonomous Unified Runtime & Agent Orchestration for multi-agent AI systems is a multi-agent AI system that automates program generation, execution, review, testing, and monitoring. It uses specialized agents that work together through linux IPC and named fifos to complete tasks efficiently. A web dashboard allows users to submit programs and monitor their execution and results. The system demonstrates how AI agents, process management, and automated runtime orchestration can be combined into a single platform

INTRODUCTION

- AURA is a multi-agent AI runtime and orchestration system.
- It uses multiple specialized agents to generate, execute, review, test, and monitor programs.
- Agents work together through linux ipc using named fifos.
- A web dashboard allows users to submit programs and monitor their execution.
- The system provides a modular, automated, and controlled execution environment.

PROBLEM STATEMENT

Traditional systems require manual management of program execution and computing resources, which can lead to inefficient resource utilization, delays, and difficulty in monitoring multiple tasks. There is a need for an intelligent system that can coordinate multiple AI agents, manage program execution, and allocate system resources efficiently. The proposed multi-agent AI execution and resource management system addresses this by using specialized agents to handle execution tasks while monitoring and managing available system resources.

OBJECTIVES

- To develop multiple autonomous AI agents for code generation, execution, review, and testing.
- To enable communication between agents using linux named pipes (fifos).
- To provide a restricted workspace for secure agent operations.
- To monitor cpu and memory usage and control agents exceeding resource limits.
- To maintain synchronization, activity logs, and backup files for reliable and controlled execution.

WORKING OF THE PROJECT

1 Step 1

User submits a task to the AI execution system.

2 Step 2

The system creates separate processes for different AI agents.

3 Step 3

Agents communicate using Named Pipes (FIFOs).

4 Step 4

The task is processed through agents such as Generator, Executor, Reviewer, and Tester.

5 Step 5

The system validates file access and provides a restricted workspace.

6 Step 6

CPU and memory usage of each agent are monitored continuously.

7 Step 7

Agents exceeding resource limits can be controlled using Linux signals.

8 Step 8

Agent activities, execution results, and file changes are logged and backed up.

OS CONCEPTS USED

Concept	Where it is used in our project
Process Management	Separate processes are created for different AI agents such as Generator, Executor, Reviewer, and Tester.
IPC – Named FIFOs	Communication between AI agents using named pipes (FIFOs).
Signals	Used to pause, resume, and stop running agent processes.
File Locking	Protects shared files during review and prevents simultaneous access.
File I/O	Stores source code, execution results, logs, and backup files in the workspace.
Process Monitoring	Monitors CPU usage, memory usage, and process state using /proc.
Synchronization	Maintains safe access to shared data and resources between agents.

SIGNALS AND LOGS

Concept	Used In Our Project	Purpose
SIGSTOP	AI-agent processes	Pauses the active process group
SIGCONT	AI-agent processes	Resumes paused processes
SIGTERM	AI-agent processes	Stops the active process group safely
SIGUSR1	C Runtime	Handles asynchronous OS events
SIGINT	C Runtime	Handles process interruption
status.json	Runtime workspace	Stores execution status, agent status, results, and monitoring data
manifest.json	Runtime workspace	Stores run information and configuration
Telemetry Events	Monitor	Records process state, CPU, and memory information

SOFTWARE REQUIREMENTS

Requirement	**Details**
Operating System	Ubuntu Linux / Ubuntu through WSL 2
Programming Languages	C and Python
Compiler	GCC with C11 support
Web Layer	Python ThreadingHTTPServer, HTML, CSS, JavaScript
IPC	Linux Named FIFOs
Synchronization	Flock and Python threading lock
Browser	Google Chrome / Microsoft Edge / Firefox
Editor	VS Code or any text editor
Build Tool	GNU Make
Runtime Utilities	Bash, ps, top, /proc, and standard Linux commands

CONCLUSION AND FUTURE SCOPE

Conclusion

- The project successfully integrates multiple AI agents with OS concepts such as processes, IPC, signals, file locking, and resource monitoring for controlled program execution.

Future scope

- Dynamic resource allocation
- More AI agents and languages
- Enhanced security and sandboxing
- Advanced monitoring and analytics

THANK YOU